# Spring Boot Auto-Configuration

7 min read · Updated 28 Jul 2026 · Topic 54 of 65

Bookmark · Mark done

## Why it matters

"How does Spring Boot know to configure a `DataSource`?" is the question that separates people who use Boot from people who understand it. The answer is not magic — it is conditional configuration plus classpath inspection, and knowing the mechanism is what lets you debug a starter that did not activate, or override one that did the wrong thing.

## The concept, simply

Auto-configuration is a large set of `@Configuration` classes shipped inside Boot, each guarded by conditions. On startup, Boot evaluates every one and applies those whose conditions match. The dominant condition is **what is on the classpath**: add the JDBC starter and a `DataSource` configuration becomes eligible; add H2 and a URL is inferred.

`@SpringBootApplication` is three annotations:

- `@SpringBootConfiguration` — this class is a configuration source.
- `@EnableAutoConfiguration` — evaluate the auto-configurations.
- `@ComponentScan` — scan this package and below.

## How it works

Boot reads candidate configuration classes from `META-INF/spring/…AutoConfiguration.imports` in every jar on the classpath (before Boot 2.7 this was `spring.factories`). Each candidate is then filtered by its conditions:

| | |
|---|---|
| `@ConditionalOnClass` | A class is present on the classpath |
| `@ConditionalOnMissingBean` | You have **not** defined that bean — the override hook |
| `@ConditionalOnProperty` | A property has a given value |
| `@ConditionalOnWebApplication` | It is a web app |

`@ConditionalOnMissingBean` **is the key to the whole design.** It is why defining your own `DataSource` bean silently replaces Boot's: the auto-configuration checks whether one already exists and backs off. You override by *declaring*, not by disabling.

## Code examples

Seeing what Boot decided, which is the actual debugging technique:

BASH Copy

```
# The condition evaluation report — every auto-config, matched or not, WIT
# the reason. This is the answer to "why is my bean not there?".
java -jar app.jar --debug

# Positive matches:  DataSourceAutoConfiguration matched:
#   - @ConditionalOnClass found 'javax.sql.DataSource'
# Negative matches:  RedisAutoConfiguration:
#   - @ConditionalOnClass did not find required class 'RedisOperations'
```

JAVA Copy

```
// Overriding: just define the bean. @ConditionalOnMissingBean backs off.
@Configuration
public class DataConfig {
    @Bean
    public DataSource dataSource() { return buildCustomPool(); }
}
```

```
// Excluding an auto-configuration entirely:
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)

// Writing your own — this is what every starter is:
@AutoConfiguration
@ConditionalOnClass(PaymentClient.class)
@EnableConfigurationProperties(PaymentProperties.class)
public class PaymentAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean            // ✓ always, so users can overrid
    public PaymentClient paymentClient(PaymentProperties props) {
        return new PaymentClient(props.url(), props.timeout());
    }
}
// Registered in:
// META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguratio
```

Type-safe configuration, which is better than scattered `@Value`:

JAVA Copy

```
@ConfigurationProperties(prefix = "payment")
@Validated
public record PaymentProperties(
        @NotBlank String url,
        @DefaultValue("5s") Duration timeout,
        @Min(1) int retries) { }

// application.yml
// payment:
//   url: https://api.example.com
//   timeout: 10s
//   retries: 3

// ✓ Validated at STARTUP — a bad value fails the build-out immediately
//   rather than at the first request. Records work as of Boot 3.
// ✗ @Value("${payment.url}") scatters configuration across the codebase
//   with no validation, no defaults and no IDE completion.
```

**Ordering matters and is explicit.** Auto-configurations run *after* user configuration — which is what makes `@ConditionalOnMissingBean` reliable — and can order among themselves with `@AutoConfigureBefore` and `@AutoConfigureAfter`. Getting this wrong in a custom starter produces the classic intermittent bug where the override works sometimes.

**The cost is startup time.** Boot evaluates every candidate auto-configuration on every start, which is a measurable part of the "why is Spring Boot slow to start?" complaint. Boot 3's AOT processing addresses this by doing the evaluation at *build* time and generating the resulting configuration — which is also what makes GraalVM native images feasible for Spring applications (Chapter 1's AOT discussion).

**Property precedence** is worth knowing because it explains "my config is being ignored". Roughly, from highest to lowest: command-line arguments, then `SPRING_APPLICATION_JSON`, then OS environment variables, then `application-{profile}.yml`, then `application.yml`, then defaults. Environment variables beating the YAML file is the one that surprises people, and it is deliberate — it is what makes twelve-factor deployment work.

## Common mistakes

- **Treating it as magic.** It is conditions plus classpath — and `--debug` prints the entire decision.
- **Excluding an auto-configuration to override it.** Usually just define the bean; the default backs off.
- **Omitting `@ConditionalOnMissingBean` in a custom starter.** Users cannot override you.
- **Components outside the scanned package.** Scanning starts at the main class's package — a bean in a sibling package is invisible.
- `@Value` **everywhere.** Use `@ConfigurationProperties` for validation, defaults and grouping.
- **Not knowing property precedence.** Environment variables beat `application.yml`.

## Interview lens

**Q: "How does Spring Boot auto-configuration work?"**
A: Boot ships many `@Configuration` classes, each guarded by conditions, and registers them in an `AutoConfiguration.imports` file in each jar. At startup it evaluates the conditions — most commonly `@ConditionalOnClass`, so behaviour follows the classpath — and applies those that match. Adding a starter puts classes on the classpath, which activates the matching configuration. Nothing is magic; `--debug` prints the full report with reasons.

**Q: "How do you override an auto-configured bean?"**
A: Usually just define your own. Auto-configurations are annotated `@ConditionalOnMissingBean` and run after user configuration, so they detect yours and back off. Excluding the auto-configuration outright is a bigger hammer and normally only needed when you want to remove a whole feature.

**Q: "What does `@SpringBootApplication` do?"**
A: It combines three: `@SpringBootConfiguration`, marking the class as a configuration source; `@EnableAutoConfiguration`, which triggers the conditional evaluation; and `@ComponentScan`, which scans the annotated class's package and everything beneath it. That last one is why placing the main class in the root package matters — a component in a sibling package is never found.

**Q: "`@Value` or `@ConfigurationProperties`?"**
A: `@ConfigurationProperties` for anything beyond a one-off. It groups related settings into a type, supports validation with `@Validated` so bad configuration fails at startup rather than at first use, gives defaults and relaxed binding, and

works with records in Boot 3. `@Value` scatters configuration keys through the codebase with no validation and no discoverability.

**Q: "Why is my bean not being picked up?"**

A: Nine times out of ten it is outside the component-scan root — scanning starts at the main class's package, so anything in a sibling package needs an explicit `@ComponentScan` or a move. Otherwise: a condition did not match, which `--debug` will tell you, or a profile excluded it, or the class has no stereotype annotation at all.

**Q: "What order do configuration sources take?"**

A: Command-line arguments beat `SPRING_APPLICATION_JSON`, which beats OS environment variables, which beat profile-specific `application-{profile}.yml`, which beats `application.yml`, which beats defaults. The one that surprises people is environment variables outranking the YAML file — which is deliberate, since it is what lets the same artefact be configured per environment.

## Cheat-card

- Auto-config = **conditional** `@Configuration` **classes** listed in `AutoConfiguration.imports` (was `spring.factories`).
- Driven mostly by `@ConditionalOnClass` → behaviour follows the classpath.
- `@ConditionalOnMissingBean` = the override hook. Define your bean; the default backs off.
- `@SpringBootApplication` = `@SpringBootConfiguration` + `@EnableAutoConfiguration` + `@ComponentScan`.
- **Scanning starts at the main class's package** — the usual "bean not found" cause.
- Debug with `--debug` — the condition evaluation report gives reasons.
- `@ConfigurationProperties` + `@Validated` over `@Value` — fails at startup.
- Precedence: **args > env vars > profile yml > application.yml**.

QUICK CHECK — no marks, just for you.

1. How do you normally override an auto-configured bean such as the DataSource?

- Just define your own bean — auto-configurations are annotated @ConditionalOnMissingBean and run after user configuration, so they detect yours and back off
- Set a property to disable auto-configuration entirely
- Subclass the auto-configuration class
- Exclude the auto-configuration class explicitly

Check answer

2. A @Component is not being picked up. What is the most likely cause?

- The application needs a restart
- It is outside the component-scan root — scanning starts at the main class's package and goes down, so a sibling package is invisible
- Its constructor is not public
- It is missing a @Bean method

Check answer

3. Which configuration source wins?

- Profile-specific YAML beats command-line arguments

- Command-line arguments beat environment variables, which beat the profile-specific YAML, which beats application.yml
- Whichever is loaded last

Check answer

‹ Previous
Bean Lifecycle, Scopes and Proxies

Next ›
Spring MVC: The Request Flow

## LEARN MORE

### ARTICLES & DOCS

- Auto-configuration — Spring Boot reference
  docs.spring.io
- Externalized configuration — Spring Boot reference
  docs.spring.io

### VIDEOS




@SpringBootApplication Internal Working | IOC Container & Dependency Injection in Spring Boot
Engineering Digest

Hand-picked references for this lesson.